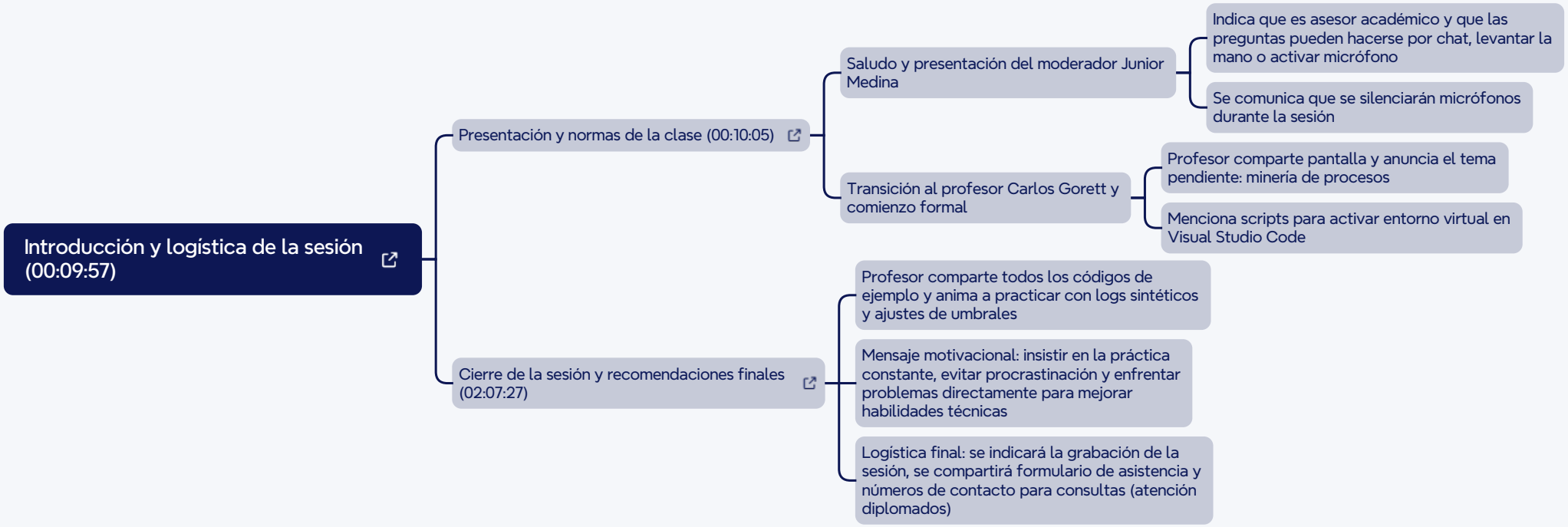
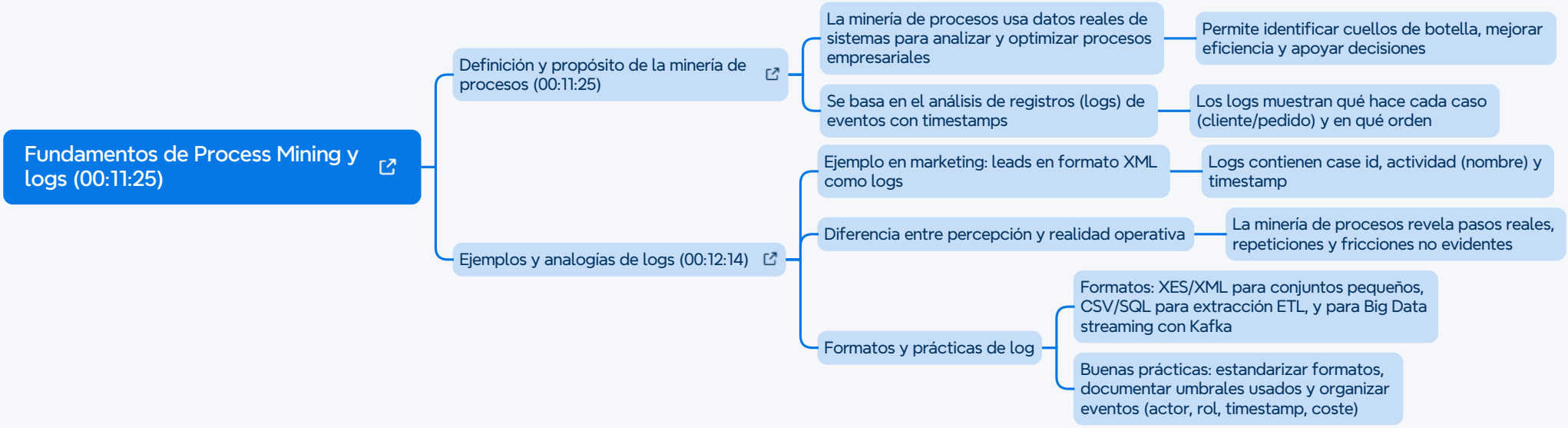
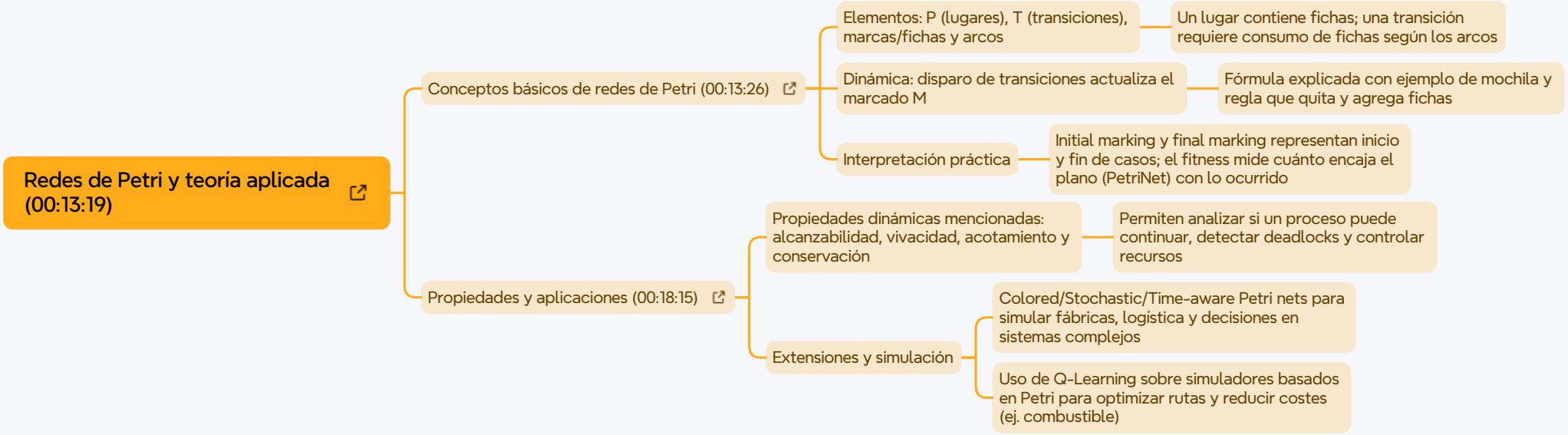
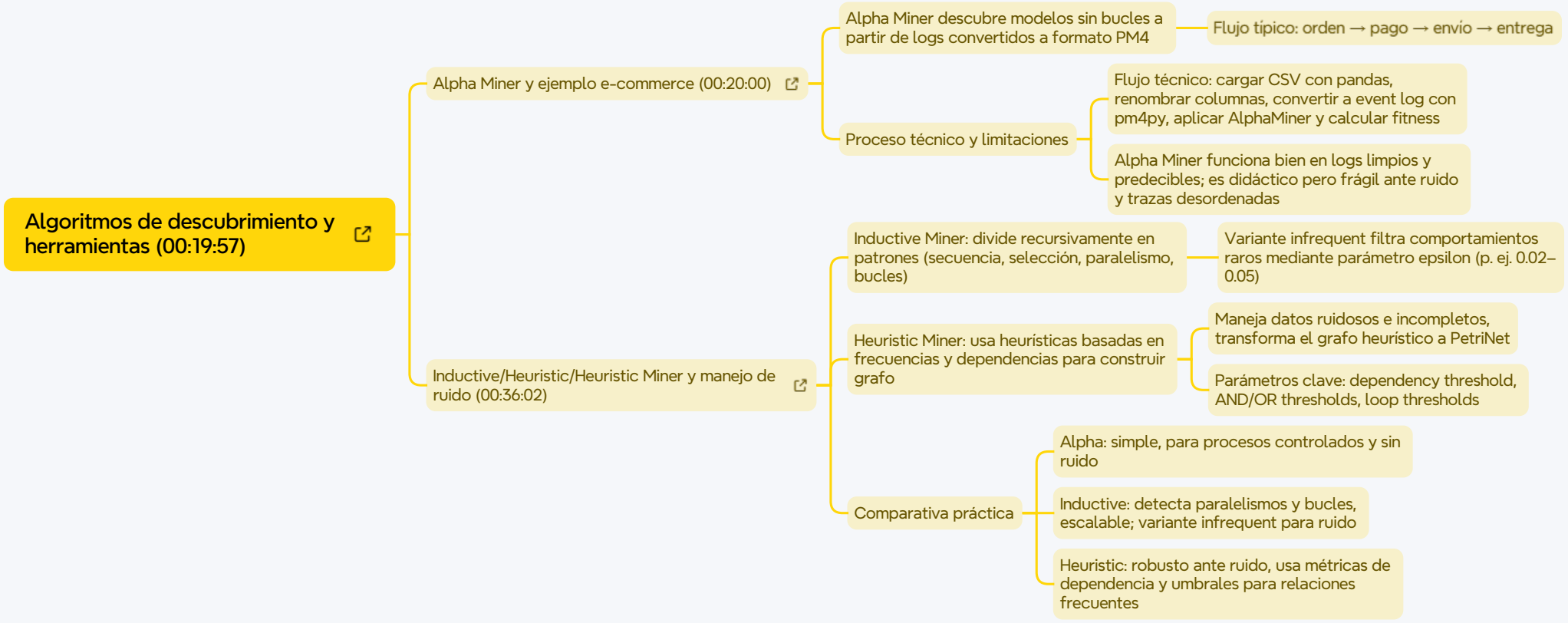
EDU MIN

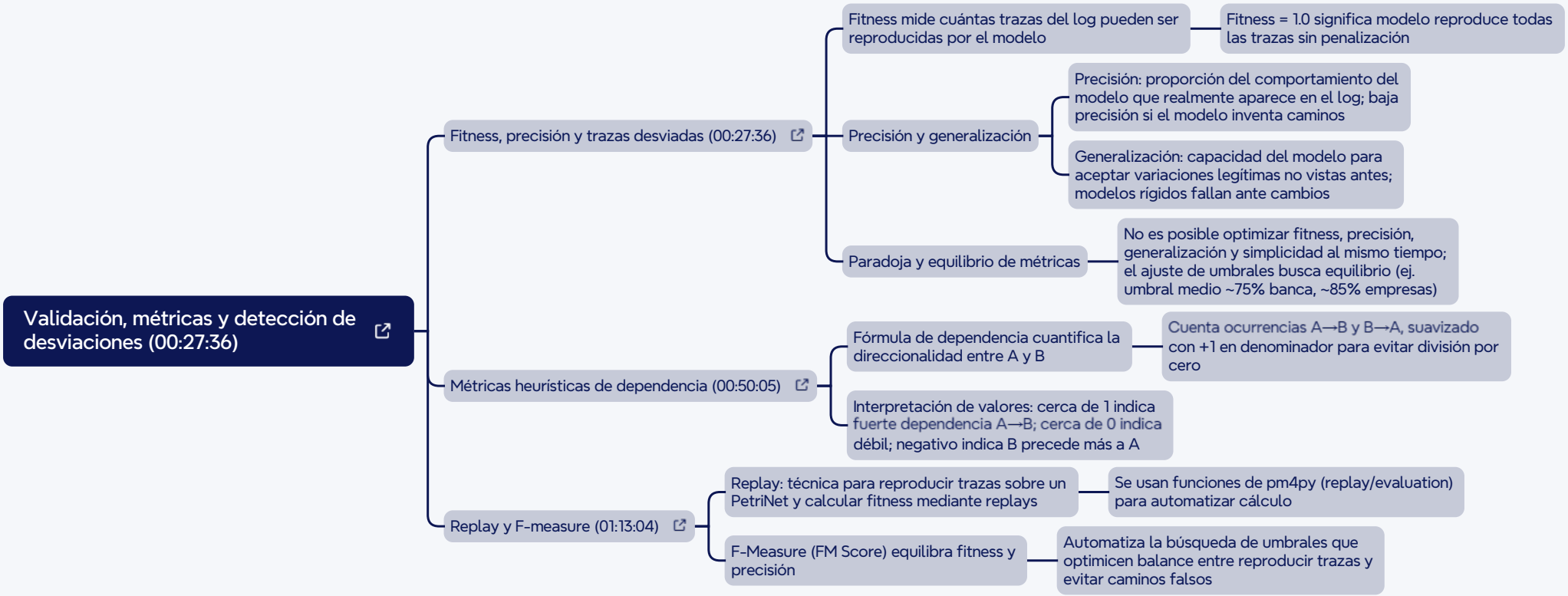

CLASE ADICIONAL: APLICACIÓN PRÁCTICA Y ZONA DE PREGUNTAS
Introducción y logística de la sesión (00:09:57)
Fundamentos de Process Mining y logs (00:11:25)
Redes de Petri y teoría aplicada (00:13:19)
Algoritmos de descubrimiento y herramientas (00:19:57)
Validación, métricas y detección de desviaciones (00:27:36)
Ventajas, limitaciones y casos de uso (00:33:47)
Buenas prácticas operativas finales (01:29:53)
Ejemplos prácticos y ejercicios en clase (01:38:19)
Integración con Machine Learning y tendencias (01:34:00)
Umbrales, estrategias de ajuste y automatización (01:02:50)
Flujo práctico, transformación de datos y herramientas (00:45:54)

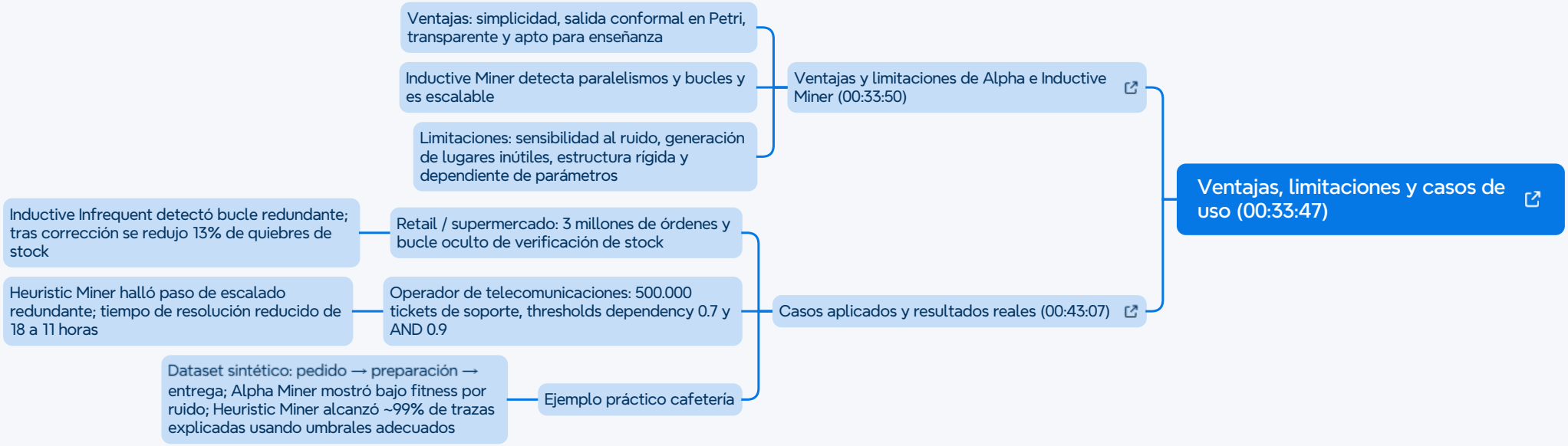

Introducción y logística de la sesión (00:09:57)
Presentación y normas de la clase (00:10:05)
Saludo y presentación del moderador Junior Medina
Indica que es asesor académico y que las preguntas pueden hacerse por chat, levantar la mano o activar micrófono
Se comunica que se silenciarán micrófonos durante la sesión
Transición al profesor Carlos Gorett y comienzo formal
Profesor comparte pantalla y anuncia el tema pendiente: minería de procesos
Menciona scripts para activar entorno virtual en Visual Studio Code
Cierre de la sesión y recomendaciones finales (02:07:27)
Profesor comparte todos los códigos de ejemplo y anima a practicar con logs sintéticos y ajustes de umbrales
Mensaje motivacional: insistir en la práctica constante, evitar procrastinación y enfrentar problemas directamente para mejorar habilidades técnicas
Logística final: se indicará la grabación de la sesión, se compartirá formulario de asistencia y números de contacto para consultas (atención diplomados)

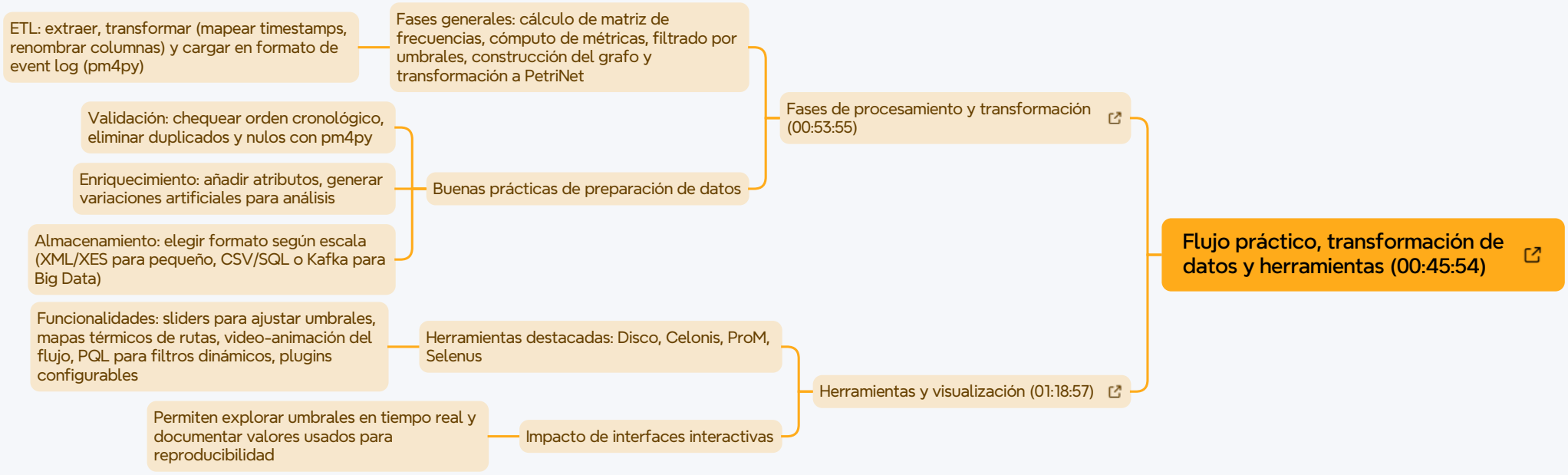

Fundamentos de Process Mining y logs (00:11:25)
Definición y propósito de la minería de procesos (00:11:25)
La minería de procesos usa datos reales de sistemas para analizar y optimizar procesos empresariales
Permite identificar cuellos de botella, mejorar eficiencia y apoyar decisiones
Se basa en el análisis de registros (logs) de eventos con timestamps
Los logs muestran qué hace cada caso (cliente/pedido) y en qué orden
Ejemplos y analogías de logs (00:12:14)
Ejemplo en marketing: leads en formato XML como logs
Logs contienen case id, actividad (nombre) y timestamp
Diferencia entre percepción y realidad operativa
La minería de procesos revela pasos reales, repeticiones y fricciones no evidentes
Formatos y prácticas de log
Formatos: XES/XML para conjuntos pequeños, CSV/SQL para extracción ETL, y para Big Data streaming con Kafka
Buenas prácticas: estandarizar formatos, documentar umbrales usados y organizar eventos (actor, rol, timestamp, coste)

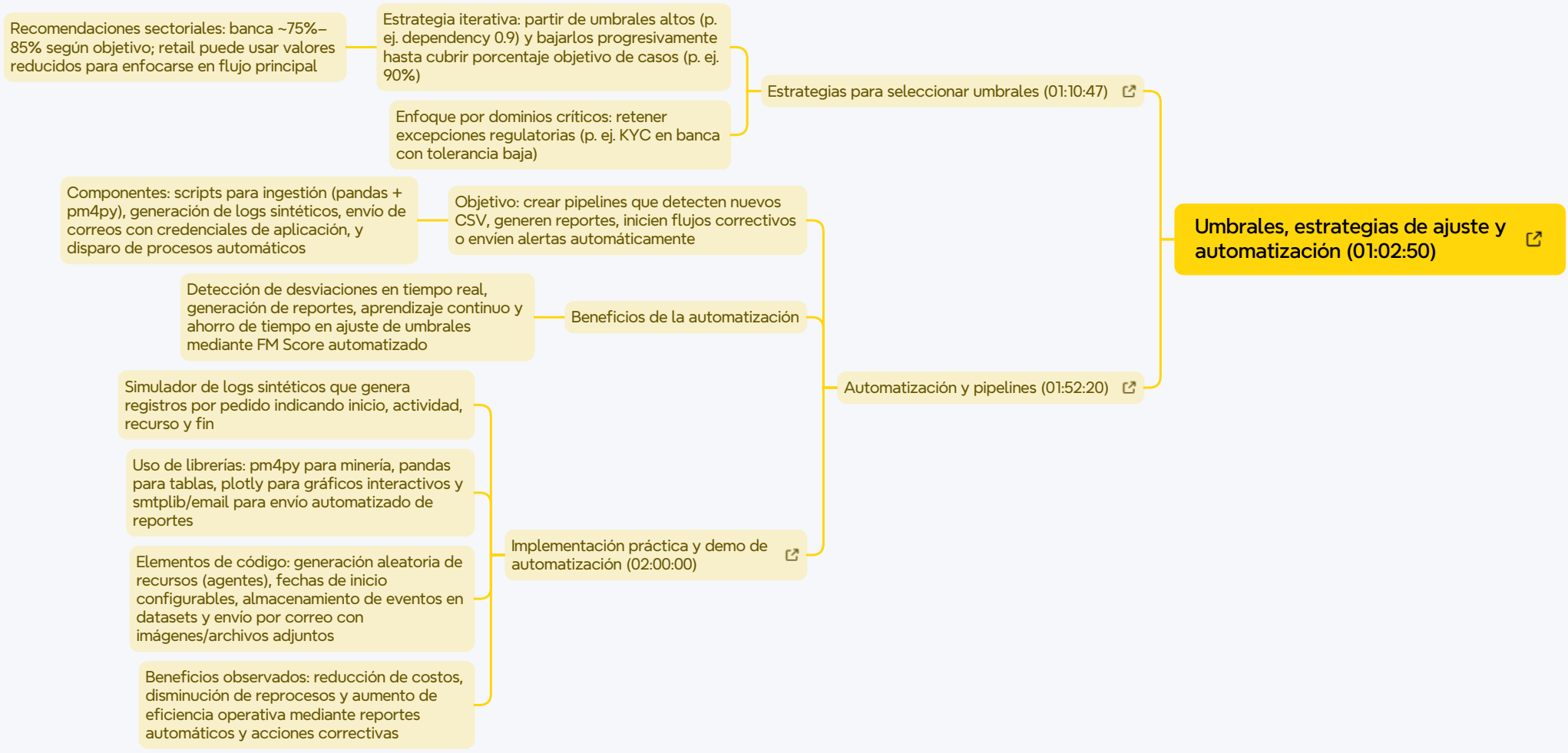

Redes de Petri y teoría aplicada (00:13:19)
Conceptos básicos de redes de Petri (00:13:26)
Elementos: P (lugares), T (transiciones), marcas/fichas y arcos
Un lugar contiene fichas; una transición requiere consumo de fichas según los arcos
Dinámica: disparo de transiciones actualiza el marcado M
Fórmula explicada con ejemplo de mochila y regla que quita y agrega fichas
Interpretación práctica
Initial marking y final marking representan inicio y fin de casos; el fitness mide cuánto encaja el plano (PetriNet) con lo ocurrido
Propiedades y aplicaciones (00:18:15)
Propiedades dinámicas mencionadas: alcanzabilidad, vivacidad, acotamiento y conservación
Permiten analizar si un proceso puede continuar, detectar deadlocks y controlar recursos
Extensiones y simulación
Colored/Stochastic/Time-aware Petri nets para simular fábricas, logística y decisiones en sistemas complejos
Uso de Q-Learning sobre simuladores basados en Petri para optimizar rutas y reducir costes (ej. combustible)

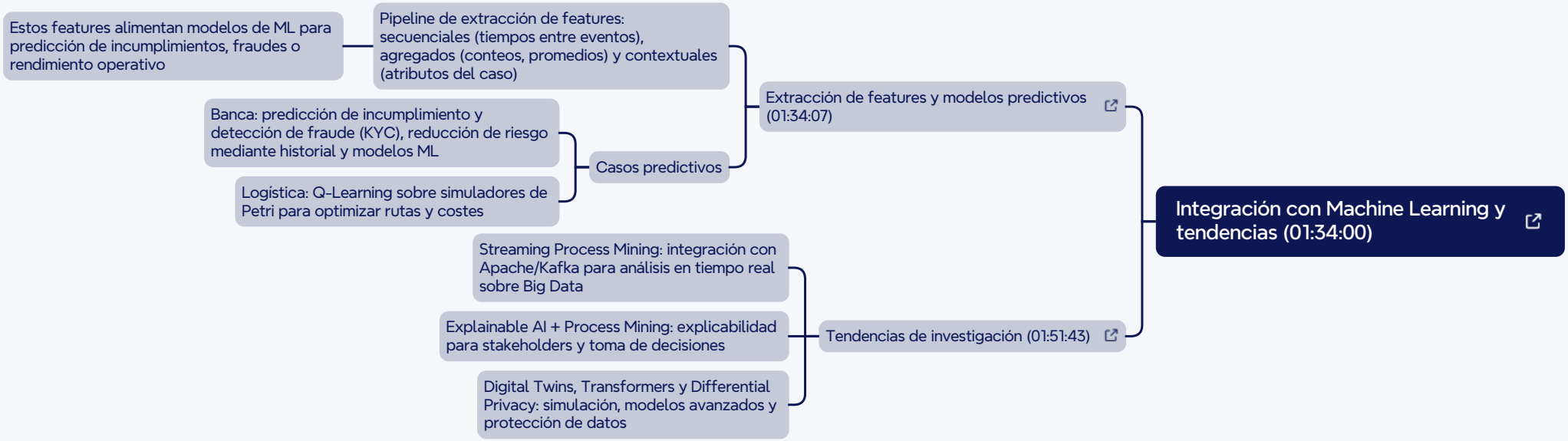

Algoritmos de descubrimiento y herramientas (00:19:57)
Alpha Miner y ejemplo e-commerce (00:20:00)
Alpha Miner descubre modelos sin bucles a partir de logs convertidos a formato PM4
Flujo típico: orden → pago → envío → entrega
Proceso técnico y limitaciones
Flujo técnico: cargar CSV con pandas, renombrar columnas, convertir a event log con pm4py, aplicar AlphaMiner y calcular fitness
Alpha Miner funciona bien en logs limpios y predecibles; es didáctico pero frágil ante ruido y trazas desordenadas
Inductive/Heuristic/Heuristic Miner y manejo de ruido (00:36:02)
Inductive Miner: divide recursivamente en patrones (secuencia, selección, paralelismo, bucles)
Variante infrequent filtra comportamientos raros mediante parámetro epsilon (p. ej. 0.02–0.05)
Heuristic Miner: usa heurísticas basadas en frecuencias y dependencias para construir grafo
Maneja datos ruidosos e incompletos, transforma el grafo heurístico a PetriNet
Parámetros clave: dependency threshold, AND/OR thresholds, loop thresholds
Comparativa práctica
Alpha: simple, para procesos controlados y sin ruido
Inductive: detecta paralelismos y bucles, escalable; variante infrequent para ruido
Heuristic: robusto ante ruido, usa métricas de dependencia y umbrales para relaciones frecuentes

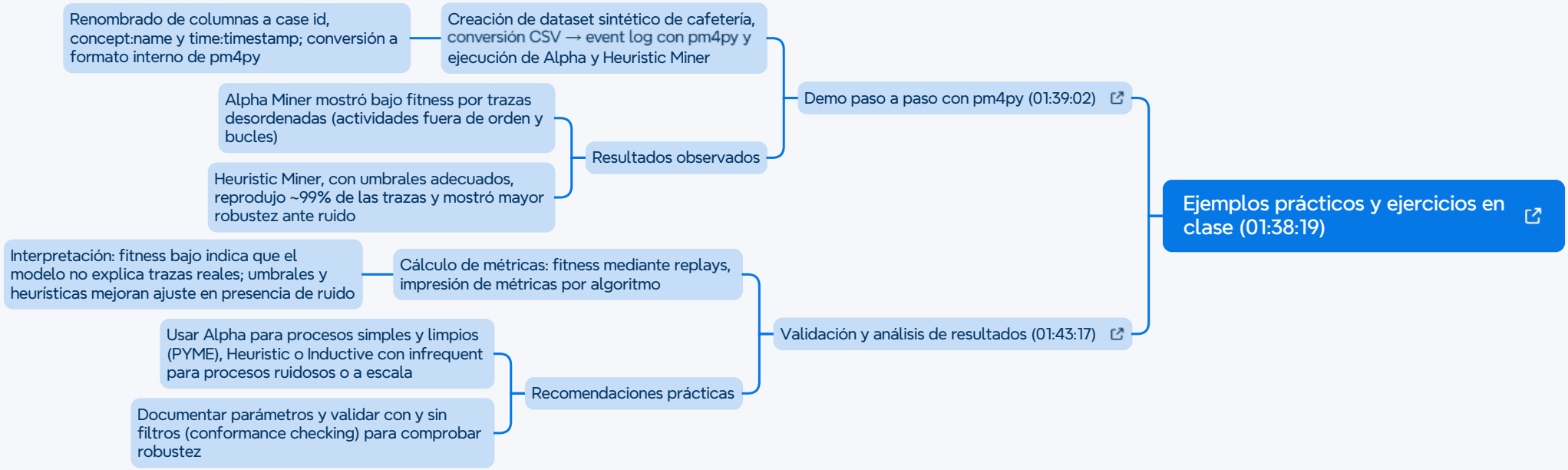

Validación, métricas y detección de desviaciones (00:27:36)
Fitness, precisión y trazas desviadas (00:27:36)
Fitness mide cuántas trazas del log pueden ser reproducidas por el modelo
Fitness = 1.0 significa modelo reproduce todas las trazas sin penalización
Precisión y generalización
Precisión: proporción del comportamiento del modelo que realmente aparece en el log; baja precisión si el modelo inventa caminos
Generalización: capacidad del modelo para aceptar variaciones legítimas no vistas antes; modelos rígidos fallan ante cambios
Paradoja y equilibrio de métricas
No es posible optimizar fitness, precisión, generalización y simplicidad al mismo tiempo; el ajuste de umbrales busca equilibrio (ej. umbral medio ~75% banca, ~85% empresas)
Métricas heurísticas de dependencia (00:50:05)
Fórmula de dependencia cuantifica la direccionalidad entre A y B
Cuenta ocurrencias A→B y B→A, suavizado con +1 en denominador para evitar división por cero
Interpretación de valores: cerca de 1 indica fuerte dependencia A→B; cerca de 0 indica débil; negativo indica B precede más a A
Replay y F-measure (01:13:04)
Replay: técnica para reproducir trazas sobre un PetriNet y calcular fitness mediante replays
Se usan funciones de pm4py (replay/evaluation) para automatizar cálculo
F-Measure (FM Score) equilibra fitness y precisión
Automatiza la búsqueda de umbrales que optimicen balance entre reproducir trazas y evitar caminos falsos

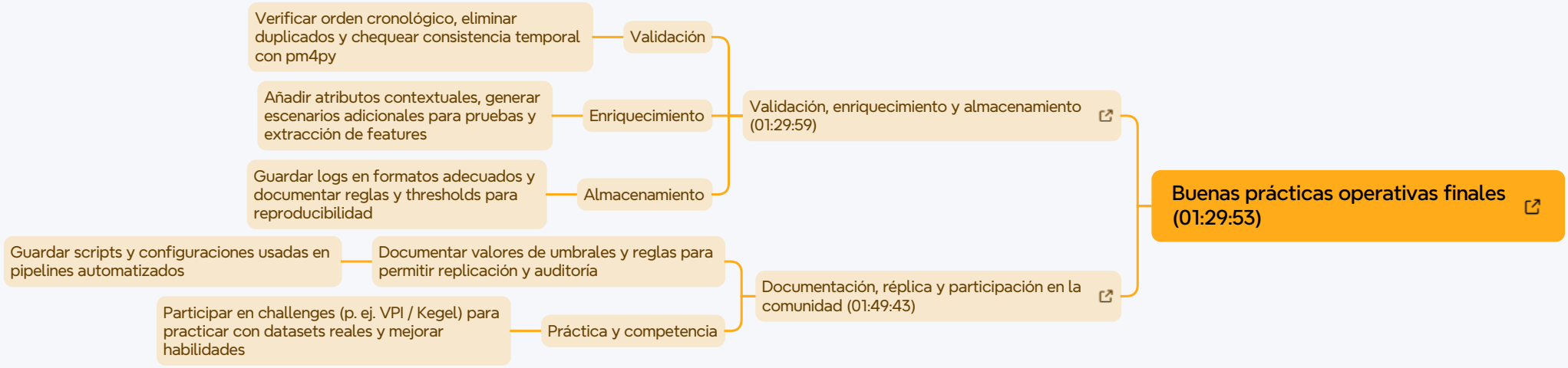

Ventajas, limitaciones y casos de uso (00:33:47)
Ventajas y limitaciones de Alpha e Inductive Miner (00:33:50)
Ventajas: simplicidad, salida conformal en Petri, transparente y apto para enseñanza
Inductive Miner detecta paralelismos y bucles y es escalable
Limitaciones: sensibilidad al ruido, generación de lugares inútiles, estructura rígida y dependiente de parámetros
Casos aplicados y resultados reales (00:43:07)
Retail / supermercado: 3 millones de órdenes y bucle oculto de verificación de stock
Inductive Infrequent detectó bucle redundante; tras corrección se redujo 13% de quiebres de stock
Operador de telecomunicaciones: 500.000 tickets de soporte, thresholds dependency 0.7 y AND 0.9
Heuristic Miner halló paso de escalado redundante; tiempo de resolución reducido de 18 a 11 horas
Ejemplo práctico cafetería
Dataset sintético: pedido → preparación → entrega; Alpha Miner mostró bajo fitness por ruido; Heuristic Miner alcanzó ~99% de trazas explicadas usando umbrales adecuados


Flujo práctico, transformación de datos y herramientas (00:45:54)
Fases de procesamiento y transformación (00:53:55)
Fases generales: cálculo de matriz de frecuencias, cómputo de métricas, filtrado por umbrales, construcción del grafo y transformación a PetriNet
ETL: extraer, transformar (mapear timestamps, renombrar columnas) y cargar en formato de event log (pm4py)
Buenas prácticas de preparación de datos
Validación: chequear orden cronológico, eliminar duplicados y nulos con pm4py
Enriquecimiento: añadir atributos, generar variaciones artificiales para análisis
Almacenamiento: elegir formato según escala (XML/XES para pequeño, CSV/SQL o Kafka para Big Data)
Herramientas y visualización (01:18:57)
Herramientas destacadas: Disco, Celonis, ProM, Selenus
Funcionalidades: sliders para ajustar umbrales, mapas térmicos de rutas, video-animación del flujo, PQL para filtros dinámicos, plugins configurables
Impacto de interfaces interactivas
Permiten explorar umbrales en tiempo real y documentar valores usados para reproducibilidad


Umbrales, estrategias de ajuste y automatización (01:02:50)
Estrategias para seleccionar umbrales (01:10:47)
Estrategia iterativa: partir de umbrales altos (p. ej. dependency 0.9) y bajarlos progresivamente hasta cubrir porcentaje objetivo de casos (p. ej. 90%)
Recomendaciones sectoriales: banca ~75%–85% según objetivo; retail puede usar valores reducidos para enfocarse en flujo principal
Enfoque por dominios críticos: retener excepciones regulatorias (p. ej. KYC en banca con tolerancia baja)
Automatización y pipelines (01:52:20)
Objetivo: crear pipelines que detecten nuevos CSV, generen reportes, inicien flujos correctivos o envíen alertas automáticamente
Componentes: scripts para ingestión (pandas + pm4py), generación de logs sintéticos, envío de correos con credenciales de aplicación, y disparo de procesos automáticos
Beneficios de la automatización
Detección de desviaciones en tiempo real, generación de reportes, aprendizaje continuo y ahorro de tiempo en ajuste de umbrales mediante FM Score automatizado
Implementación práctica y demo de automatización (02:00:00)
Simulador de logs sintéticos que genera registros por pedido indicando inicio, actividad, recurso y fin
Uso de librerías: pm4py para minería, pandas para tablas, plotly para gráficos interactivos y smtplib/email para envío automatizado de reportes
Elementos de código: generación aleatoria de recursos (agentes), fechas de inicio configurables, almacenamiento de eventos en datasets y envío por correo con imágenes/archivos adjuntos
Beneficios observados: reducción de costos, disminución de reprocesos y aumento de eficiencia operativa mediante reportes automáticos y acciones correctivas


Integración con Machine Learning y tendencias (01:34:00)
Extracción de features y modelos predictivos (01:34:07)
Pipeline de extracción de features: secuenciales (tiempos entre eventos), agregados (conteos, promedios) y contextuales (atributos del caso)
Estos features alimentan modelos de ML para predicción de incumplimientos, fraudes o rendimiento operativo
Casos predictivos
Banca: predicción de incumplimiento y detección de fraude (KYC), reducción de riesgo mediante historial y modelos ML
Logística: Q-Learning sobre simuladores de Petri para optimizar rutas y costes
Tendencias de investigación (01:51:43)
Streaming Process Mining: integración con Apache/Kafka para análisis en tiempo real sobre Big Data
Explainable AI + Process Mining: explicabilidad para stakeholders y toma de decisiones
Digital Twins, Transformers y Differential Privacy: simulación, modelos avanzados y protección de datos


Ejemplos prácticos y ejercicios en clase (01:38:19)
Demo paso a paso con pm4py (01:39:02)
Creación de dataset sintético de cafetería, conversión CSV → event log con pm4py y ejecución de Alpha y Heuristic Miner
Renombrado de columnas a case id, concept:name y time:timestamp; conversión a formato interno de pm4py
Resultados observados
Alpha Miner mostró bajo fitness por trazas desordenadas (actividades fuera de orden y bucles)
Heuristic Miner, con umbrales adecuados, reprodujo ~99% de las trazas y mostró mayor robustez ante ruido
Validación y análisis de resultados (01:43:17)
Cálculo de métricas: fitness mediante replays, impresión de métricas por algoritmo
Interpretación: fitness bajo indica que el modelo no explica trazas reales; umbrales y heurísticas mejoran ajuste en presencia de ruido
Recomendaciones prácticas
Usar Alpha para procesos simples y limpios (PYME), Heuristic o Inductive con infrequent para procesos ruidosos o a escala
Documentar parámetros y validar con y sin filtros (conformance checking) para comprobar robustez


Buenas prácticas operativas finales (01:29:53)
Validación, enriquecimiento y almacenamiento (01:29:59)
Validación
Verificar orden cronológico, eliminar duplicados y chequear consistencia temporal con pm4py
Enriquecimiento
Añadir atributos contextuales, generar escenarios adicionales para pruebas y extracción de features
Almacenamiento
Guardar logs en formatos adecuados y documentar reglas y thresholds para reproducibilidad
Documentación, réplica y participación en la comunidad (01:49:43)
Documentar valores de umbrales y reglas para permitir replicación y auditoría
Guardar scripts y configuraciones usadas en pipelines automatizados
Práctica y competencia
Participar en challenges (p. ej. VPI / Kegel) para practicar con datasets reales y mejorar habilidades